

UNIVERSITÉ MOHAMMED PREMIER
École Nationale des Sciences Appliquées d'Oujda
Filière Génie Informatique – GI3

RAPPORT DE MINI-PROJET
Module : Développement Java IHM

HotelManager

Application de gestion hôtelière — chambres & réservations

Réalisé par :
Mazen EL ALLALI
Zouhir ATTANOUTI

Encadrante :
Mme Douae EL HILA

Année universitaire : 2025 / 2026

Remerciements

À Mme Douae EL HILA et Mme Ilhame EL FARISSI

Au terme de ce mini-projet, nous tenons à exprimer notre profonde gratitude pour l'opportunité d'évoluer nos compétences académiques et de relever de nouveaux défis tout au long du processus, ainsi que pour la chance de nous familiariser avec ce type de travail, qui sera primordial de maîtriser pour les années à venir, Inchallah.

Nous remercions tout particulièrement notre encadrante, Mme Douae EL HILA. Sa rigueur et son aide constante nous ont inspirés à donner le meilleur de nous-mêmes. Elle n'a jamais cessé de se rendre disponible pour nous aider, quel que soit le problème rencontré, et ses recommandations ont été précieuses pour orienter notre démarche et améliorer la qualité de notre application.

Nous adressons également nos sincères remerciements à Mme Ilhame EL FARISSI pour la formation qu'elle nous a dispensée tout au long de l'année. L'environnement amical qu'elle établit avec nous facilite réellement l'apprentissage et la compréhension : on ressort de chacune de ses séances satisfait et avec l'envie d'aller plus loin.

Enfin, nous espérons que ce travail reflète, ne serait-ce qu'en partie, les efforts qu'elles ont consacrés à notre égard et qu'elle réponde à vos attentes.

Table des matières

Introduction	6
Chapitre 1 : Présentation du projet	7
1.1 Description de l'application	7
1.2 Besoins fonctionnels	7
1.3 Besoins non fonctionnels	7
Chapitre 2 : Conception	9
2.1 Architecture MVC	9
2.2 Diagramme de classes UML	9
2.3 Diagramme de cas d'utilisation	10
2.4 Diagramme de séquence	11
Chapitre 3 : Environnement de travail	12
3.1 Outils et technologies utilisés	12
3.2 Structure du projet	12
Chapitre 4 : Répartition des tâches	14
4.1 Tâches de l'étudiant 1 - El Allali Mazen	15
4.2 Tâches de l'étudiant 2 - Attanouti Zouhir	15
Chapitre 5 : Explication du code	16
5.1 Connexion à la base de données - Database.java	16
5.2 Les classes modèles	18
5.3 Les classes DAO	19
5.4 Les contrôleurs	20
5.5 Les fichiers FXML	22
5.6 Fonctionnalité spécifique au projet	23
Chapitre 6 : Interfaces de l'application	26
6.1 Fenêtre principale	26
6.2 Interfaces de gestion de l'entité 1 (Chambres)	26
6.3 Interfaces de gestion de l'entité 2 (Réservations)	27

6.4 Tableau de bord - Statistiques	28
6.5 Menus et dialogues	29
6.6 Export de données	29
6.7 Génération de facture	30
Conclusion	31
Références	32

Table des figures

Figure 2.1 – Architecture en couches de l’application (MVC + DAO)	9
Figure 2.2 – Diagramme de classes du domaine et de la couche DAO	10
Figure 2.3 – Diagramme de cas d’utilisation	10
Figure 2.4 – Diagramme de séquence du scénario	11
Figure 6.1 – Fenêtre principale de l’application	26
Figure 6.2.1 – Gestion des chambres	26
Figure 6.2.2 – Gestion des chambres	27
Figure 6.3.1 – Gestion des réservations	27
Figure 6.3.2 – Gestion des réservations	27
Figure 6.4.1 – Tableau de bord et statistiques	28
Figure 6.4.2 – Tableau de bord et statistiques	28
Figure 6.5.1 – Menus et dialogues	29
Figure 6.6 – Export des données au format CSV	29
Figure 6.7 – Génération d’une facture	30

Liste des tableaux

Tableau 3.1 – Outils et technologies utilisés	12
Tableau 3.2 – Rôle des différentes couches	13
Tableau 4.1 – Tâches réalisées par Mazen El Allali	15
Tableau 4.2 – Tâches réalisées par Zouhir Attanouti	15

Introduction

La gestion quotidienne d'un établissement hôtelier repose sur deux activités étroitement liées : l'administration du parc de chambres et le suivi des réservations des clients. Réalisées manuellement, ces tâches deviennent vite source d'erreurs (double réservation, perte d'information, calculs de tarifs approximatifs) et de perte de temps. L'informatisation de ces processus apporte une réponse concrète à ce besoin de fiabilité et de rapidité.

C'est dans ce cadre que s'inscrit notre mini-projet du module « Développement Java IHM ». Il consiste à concevoir et développer une application de bureau complète, dotée d'une interface graphique moderne réalisée avec *JavaFX* et connectée à une base de données *MySQL*. Nous avons choisi le domaine de la gestion hôtelière car il met naturellement en jeu deux entités principales reliées entre elles — la Chambre et la Réservation — ce qui correspond parfaitement aux exigences du sujet.

Notre application, baptisée *HotelManager*, permet de gérer le cycle de vie complet des chambres et des réservations : ajout, modification, suppression, recherche, filtrage, visualisation de statistiques et export des données. Elle a été développée en respectant une architecture logicielle rigoureuse (MVC associée au pattern DAO), gage de lisibilité, de maintenabilité et d'évolutivité.

Ce rapport rend compte de l'ensemble de notre travail. Il présente d'abord le projet et ses besoins, puis détaille la conception (architecture et diagrammes UML), l'environnement de travail, la répartition des tâches au sein du binôme, l'explication approfondie du code et, enfin, les interfaces de l'application. Une conclusion qui dresse le bilan et ouvre des perspectives d'amélioration.

Chapitre 1 Présentation du projet

1.1 Description de l'application

HotelManager est une application de bureau destinée à la gestion d'un hôtel de taille petite à moyenne. Elle s'organise autour de deux entités métier principales :

- la **Chambre** (entité 1) : décrite par un numéro, un type (Simple, Double, Suite, Familiale, Deluxe), un prix par nuit, une capacité, un étage, un état de disponibilité et une description ;
- la **Réservation** (entité 2) : rattachée à une chambre par une clé étrangère, elle enregistre les informations du client (nom, e-mail, téléphone), les dates d'arrivée et de départ, le nombre de personnes, le statut (En attente, Confirmée, Annulée, Terminée), le montant total, l'état du paiement et d'éventuelles remarques.

L'interface est organisée en sections claires accessibles depuis une barre de navigation latérale : gestion des chambres, gestion des réservations et tableau de bord statistique. Chaque réservation est obligatoirement liée à une chambre existante, ce qui garantit la cohérence des données. L'application calcule automatiquement le montant d'une réservation (nombre de nuits multiplié par le prix de la chambre), propose une recherche instantanée, des filtres, des graphiques de synthèse, l'export CSV des données ainsi que la génération de factures.

1.2 Besoins fonctionnels

Les besoins fonctionnels décrivent ce que l'application doit être capable de faire. Nous les avons regroupés comme suit :

1. **Gestion des chambres** : ajouter, modifier et supprimer une chambre via un formulaire dédié.
2. **Gestion des réservations** : créer, éditer et supprimer une réservation en la liant à une chambre, avec calcul automatique du montant.
3. **Tableau récapitulatif** : afficher l'ensemble des données dans des TableView clairs et triables.
4. **Recherche et filtrage** : retrouver instantanément un élément par mot-clé et filtrer par type de chambre ou par statut de réservation.
5. **Statistiques** : présenter des indicateurs chiffrés (nombre de chambres, disponibilités, chiffre d'affaires) et des graphiques (répartition par type, par statut, taux d'occupation).
6. **Export de données** : exporter les listes au format CSV à l'aide d'un FileChooser.
7. **Fonctionnalités complémentaires** : génération et export de factures, personnalisation du thème et bascule clair/sombre.

1.3 Besoins non fonctionnels

Au-delà des fonctionnalités, l'application doit respecter un ensemble de qualités attendues :

- **Ergonomie** : interface intuitive, cohérente et agréable, guidée par des info-bulles (Tooltip) et des messages clairs.
- **Fiabilité** : validation systématique des saisies (champs obligatoires, format d'e-mail, cohérence des dates) et confirmation des actions sensibles par des boîtes de dialogue.

- **Performance** : affichage fluide et requêtes optimisées (jointures SQL et index) pour éviter les ralentissements.
- **Maintenabilité** : code organisé selon l'architecture MVC/DAO, où chaque fichier possède une responsabilité unique et bien définie.
- **Sécurité** : utilisation de requêtes préparées (PreparedStatement) pour se prémunir des injections SQL.
- **Portabilité** : projet géré par Maven, exécutable sur tout poste disposant d'un JDK et d'un serveur MySQL.

Chapitre 2 Conception

La phase de conception a précédé l'écriture du code. Elle nous a permis de définir clairement l'architecture de l'application, d'identifier les entités et leurs relations, puis de modéliser le tout à l'aide de diagrammes UML avant de concevoir le schéma de la base de données.

2.1 Architecture MVC

Nous avons adopté l'architecture MVC (Modèle - Vue - Contrôleur), renforcée par le pattern DAO (Data Access Object) pour isoler l'accès aux données. Cette séparation des responsabilités constitue le socle de notre projet :

- **Modèle (Model)** : les classes Java qui représentent les données métier (Chambre, Reservation) et les types énumérés associés.
- **Vue (View)** : les fichiers FXML décrivant l'interface de manière déclarative, accompagnés d'une feuille de style CSS.
- **Contrôleur (Controller)** : les classes qui relient la vue au modèle, gèrent les événements de l'utilisateur, valident les saisies et déclenchent les opérations.
- **DAO** : une couche intermédiaire dédiée au dialogue avec MySQL via JDBC ; elle traduit les objets Java en requêtes SQL et inversement.

Concrètement, lorsque l'utilisateur agit sur l'interface (la Vue), le Contrôleur intercepte l'événement, sollicite le DAO approprié qui interroge la base, met à jour le Modèle, puis rafraîchit la Vue. Ce découplage permet de modifier une couche sans impacter les autres.

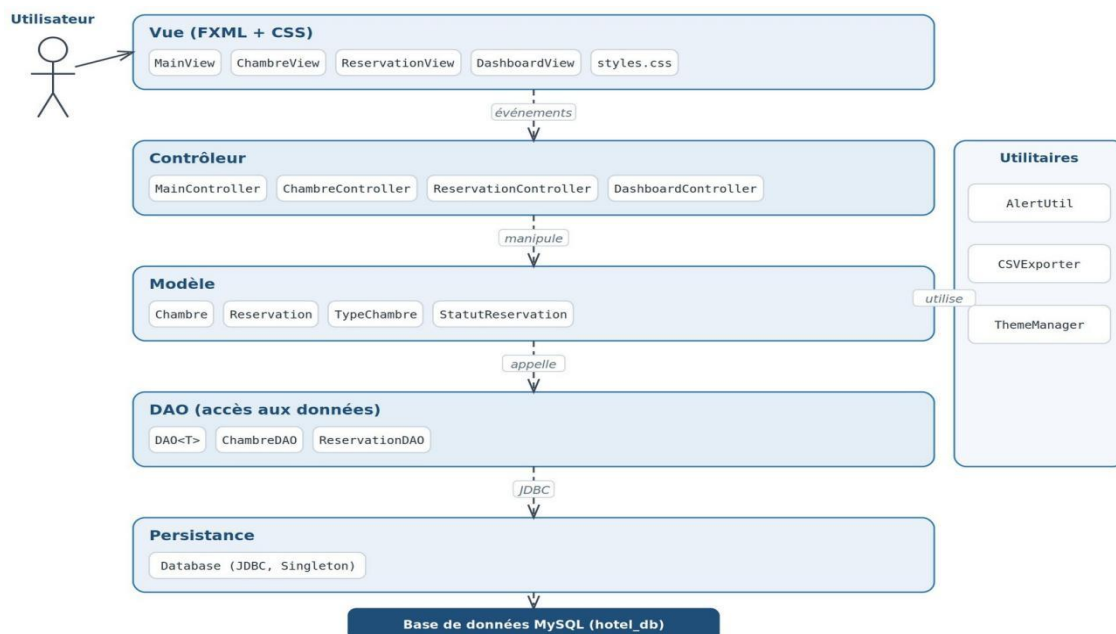


Figure 2.1 — Architecture en couches de l'application (MVC + DAO)

2.2 Diagramme de classes UML

Le diagramme de classes modélise les entités du domaine, leurs attributs, leurs méthodes et leurs relations. On y retrouve notamment la relation entre `Chambre` et `Reservation` (une chambre peut faire l'objet de plusieurs réservations), ainsi que les énumérations `TypeChambre` et `StatutReservation`. Il est présenté ci-dessous :

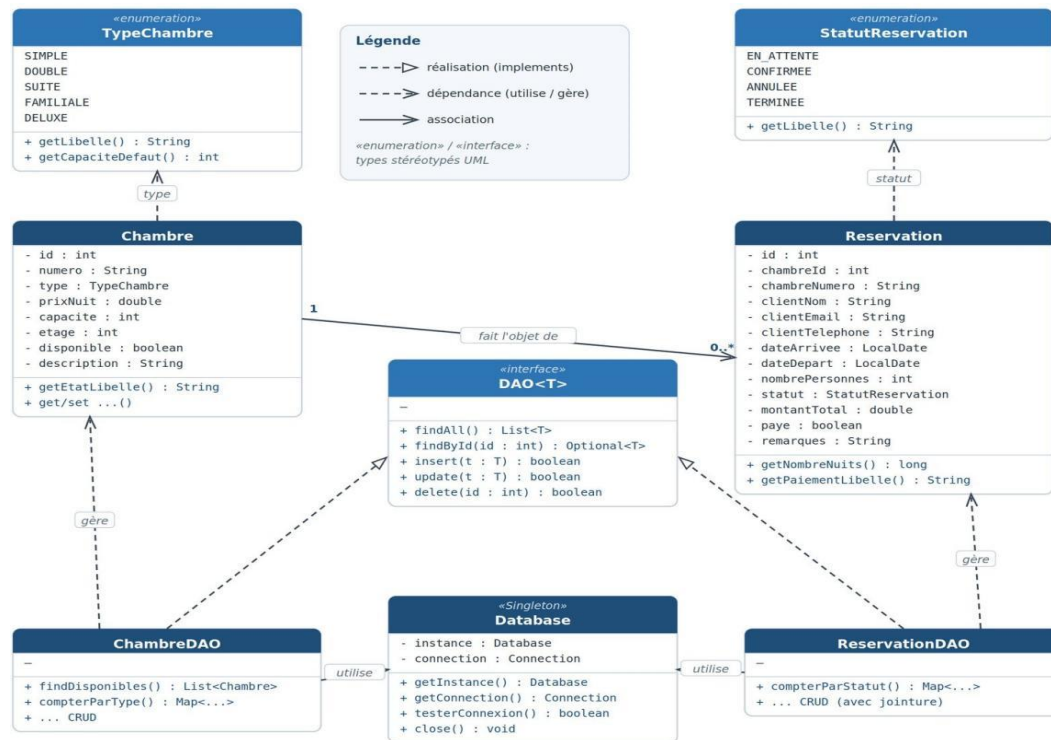


Figure 2.2 — Diagramme de classes du domaine et de la couche DAO

2.3 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation représente les interactions entre l'acteur principal (le réceptionniste / gestionnaire) et les fonctionnalités du système : gérer les chambres, gérer les réservations, consulter les statistiques, exporter les données, etc. Il est présenté ci-dessous :

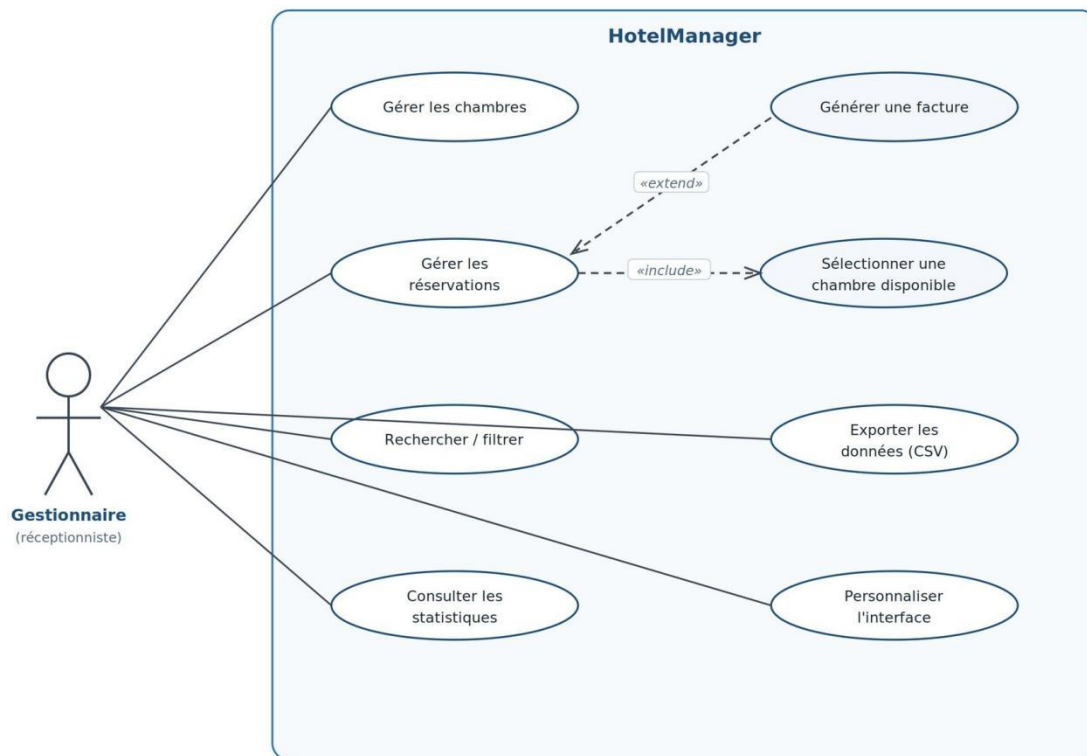


Figure 2.3 — Diagramme de cas d'utilisation

2.4 Diagramme de séquence

Le diagramme de séquence illustre le déroulement temporel d'un scénario type, par exemple « Créer une réservation » : sélection d'une chambre, saisie des informations, validation, calcul du montant, enregistrement en base et rafraîchissement du tableau. Il est présenté ci-dessous :

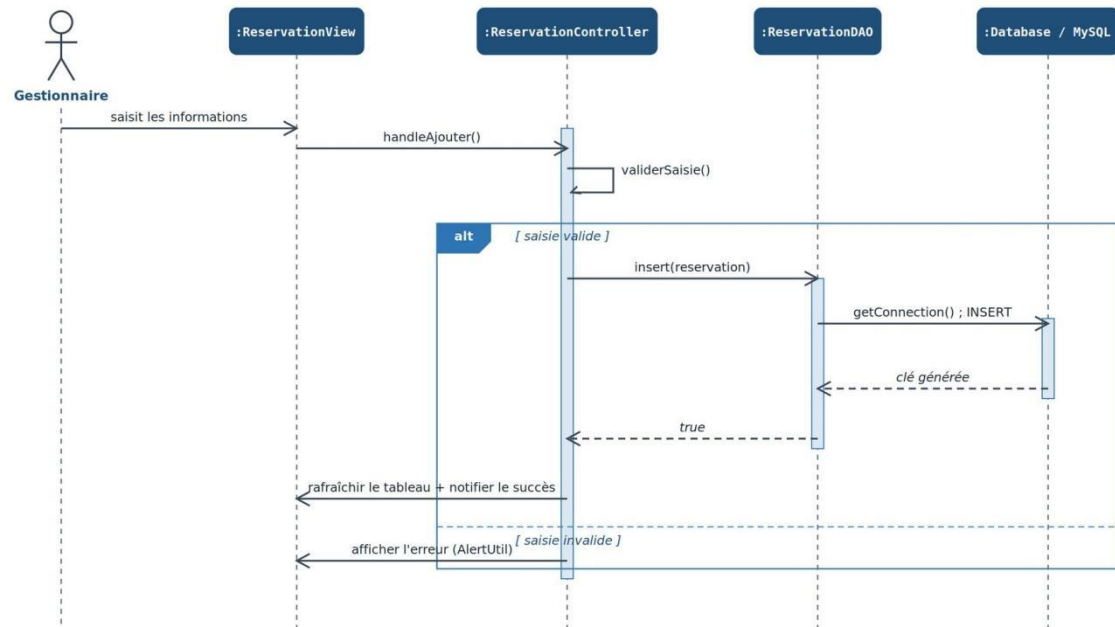


Figure 2.4 — Diagramme de séquence du scénario « Créer une réservation »

Chapitre 3 Environnement de travail

3.1 Outils et technologies utilisés

La réalisation de l'application a mobilisé un ensemble cohérent d'outils et de technologies, choisis pour leur fiabilité et leur adéquation avec les exigences du module :

Outil / Technologie	Version	Rôle dans le projet
Java (JDK)	17	Langage de programmation et plateforme d'exécution.
JavaFX	21	Bibliothèque de construction de l'interface graphique (contrôles, FXML, CSS).
MySQL	8.x	Système de gestion de base de données relationnelle (stockage des chambres et réservations).
WampServer	MySQL 9.1.0	Environnement local fournissant le serveur MySQL et phpMyAdmin.
JDBC	mysql-connector-j 8.3	Pilote assurant la communication entre Java et MySQL.
Apache Maven	3.8+	Gestion des dépendances et automatisation de la compilation/exécution.
IntelliJ IDEA / VS Code	—	Environnements de développement (édition, débogage).
Scene Builder	—	Conception visuelle des vues FXML.
Git & GitHub	—	Gestion de versions et travail collaboratif en binôme.
phpMyAdmin	—	Administration de la base et exécution du script init_db.sql.

Tableau 3.1 — Outils et technologies utilisés

3.2 Structure du projet

Le projet suit l'organisation standard d'un projet Maven, avec une séparation nette des couches conforme à l'architecture MVC/DAO. L'arborescence est la suivante :

```

HotelManagerFX/
├── pom.xml                # Dépendances Maven + plugin d'exécution
├── init_db.sql            # Création de la base + données d'exemple
├── README.md              # Présentation, prérequis, lancement
├── src/main/
│   ├── java/ma/ensao/hotel/
│   │   ├── MainApp.java    # Point d'entrée JavaFX
│   │   └── model/          # Entités + énumérations
│   │       ├── Chambre.java
│   │       ├── Reservation.java
│   │       ├── TypeChambre.java
│   │       └── StatutReservation.java
│   ├── dao/                # Accès aux données (JDBC)
│   │   ├── Database.java    # Connexion MySQL (Singleton)
│   │   ├── DAO.java         # Interface CRUD générique
│   │   ├── ChambreDAO.java
│   │   └── ReservationDAO.java
│   └── controller/         # Contrôleurs des vues

```

```

├── MainController.java
├── ChambreController.java
├── ReservationController.java
├── DashboardController.java
├── util/                # Services transverses
│   ├── AlertUtil.java   # Boîtes de dialogue
│   ├── CSVExporter.java  # Export CSV
│   └── ThemeManager.java # Thème dynamique
├── resources/
│   ├── database.properties # Identifiants de connexion
│   └── ma/ensao/hotel/
│       ├── view/         # Fichiers FXML
│       │   ├── MainView.fxml
│       │   ├── ChambreView.fxml
│       │   ├── ReservationView.fxml
│       │   └── DashboardView.fxml
│       └── css/styles.css # Feuille de style

```

Chaque dossier porte une responsabilité unique, ce qui rend le projet facile à comprendre et à faire évoluer :

Couche	Responsabilité
model	Représente les données métier (chambres, réservations) et les types énumérés.
dao	Dialogue avec MySQL : requêtes SQL et conversion ResultSet ↔ objets Java.
controller	Relie les vues à la logique : événements, validation, mise à jour de l'affichage.
view (FXML)	Décrit l'interface graphique de façon déclarative ; le CSS en assure le style.
util	Services réutilisables : alertes, export CSV, gestion du thème.

Tableau 3.2 — Rôle des différentes couches

Chapitre 4 Répartition des tâches

Le projet a été mené en binôme, dans un esprit professionnel fondé sur trois principes : une démarche méthodique et progressive, une séparation claire des responsabilités, et une communication continue. Avant de présenter la répartition individuelle, nous décrivons la démarche d'ensemble que nous avons suivie, telle que l'aurait conduite une véritable équipe de développement.

Démarche de développement adoptée

Plutôt que de coder « au fil de l'eau », nous avons structuré le travail en étapes successives, chacune validée avant de passer à la suivante. Cette progression, inspirée des méthodes itératives, nous a permis de garder le contrôle sur la qualité à chaque phase :

1. **Cadrage et choix du sujet** : Nous avons d'abord choisi le domaine (gestion hôtelière), identifié les deux entités principales (Chambre et Réservation) et leur relation.
2. **Conception** : Nous avons listé les attributs de chaque entité, élaboré les diagrammes UML (classes, cas d'utilisation, séquence) puis traduit le modèle en un schéma de base de données relationnel, avec clé étrangère et index.
3. **Mise en place du squelette** : Création du projet Maven, définition de l'arborescence des paquets (model, dao, controller, util, view), configuration du pom.xml et initialisation du dépôt Git partagé.
4. **Couche modèle** : Écriture des classes POJO (Chambre, Reservation) et des énumérations, en prévoyant dès le départ les getters nécessaires à l'affichage dans les tableaux.
5. **Couche d'accès aux données** : Implémentation de Database.java (connexion Singleton), de l'interface générique DAO<T> et des DAO concrets, avec des requêtes préparées et une jointure pour les réservations.
6. **Construction des vues** : Conception des fichiers FXML (avec BorderPane, VBox, HBox, GridPane) et de la feuille de style CSS, en intégrant l'ensemble des contrôles JavaFX exigés.
7. **Contrôleurs et liaison** : Connexion des vues à la logique : remplissage des tableaux, validation des formulaires, branchement des boutons sur les méthodes DAO et synchronisation des onglets.
8. **Fonctionnalités avancées** : Ajout de la recherche/filtrage, du tableau de bord statistique (graphiques, indicateurs), de l'export CSV, de la génération de factures et de la personnalisation du thème.
9. **Tests et débogage** : Vérification de toutes les opérations CRUD et des cas limites (champs vides, dates incohérentes, suppression d'un enregistrement lié).
10. **Documentation et livrables** : Rédaction du README, commentaires du code, préparation du rapport et enregistrement de la vidéo de démonstration.

Tout au long de ces étapes, le travail a été partagé de façon équilibrée et coordonné via GitHub (commits réguliers, messages explicites, relectures croisées). Les trois livrables — le code source, le présent rapport et la vidéo de démonstration — ont ainsi été préparés de manière sérieuse, bien organisée et soigneusement communiquée entre nous .

4.1 Tâches de l'étudiant 1 – El Allali Mazen

Tâche réalisée	Fichier(s) concerné(s)	Statut
Conception du schéma de la base de données et script d'initialisation	init_db.sql	Réalisé
Couche d'accès aux données (connexion et DAO)	Database.java, DAO.java, ChambreDAO.java, ReservationDAO.java	Réalisé
Modélisation des entités métier	Chambre.java, Reservation.java, TypeChambre.java, StatutReservation.java	Réalisé
Gestion de l'entité Chambre (formulaire et CRUD)	ChambreView.fxml, ChambreController.java	Réalisé
Export CSV des données	CSVExporter.java	Réalisé
Configuration Maven et tests de la base	pom.xml, database.properties	Réalisé
Rédaction du plan et de la structure du rapport	Rapport_HotelManager.docx	Réalisé

Tableau 4.1 — Tâches réalisées par Mazen EL Allali

4.2 Tâches de l'étudiant 2 – Attanouti Zouhir

Tâche réalisée	Fichier(s) concerné(s)	Statut
Conception de l'interface principale et navigation	MainView.fxml, MainController.java	Réalisé
Gestion de l'entité Réservation (lien avec la chambre, calcul du montant)	ReservationView.fxml, ReservationController.java	Réalisé
Tableau de bord : indicateurs et graphiques	DashboardView.fxml, DashboardController.java	Réalisé
Feuille de style, thème dynamique et mode sombre	styles.css, ThemeManager.java	Réalisé
Boîtes de dialogue, notifications et génération de factures	AlertUtil.java, ReservationController.java	Réalisé
Rédaction du README et de la documentation	README.md	Réalisé
Création de la vidéo de démonstration (1 à 2 minutes)	Vidéo de démonstration (Google Drive)	Réalisé

Tableau 4.2 — Tâches réalisées par Zouhir Attanouti

Cette répartition reflète une responsabilité dominante pour chaque membre, mais l'essentiel des décisions de conception, des relectures et des phases de test a été mené en commun, conformément à l'esprit d'un travail d'équipe.

Chapitre 5 Explication du code

Ce chapitre constitue le cœur technique du rapport : il explique le fonctionnement interne de l'application et démontre notre maîtrise du code. Pour en rendre la lecture claire et homogène, chaque partie suit la même trame en cinq temps : le contexte (le rôle de la partie), la liste des classes concernées accompagnées de leur rôle précis, le fonctionnement, les choix architecturaux, puis les extraits de code les plus significatifs.

Concernant la présentation des classes, nous avons choisi de les regrouper dans un tableau récapitulatif plutôt que de décrire chacune dans un paragraphe séparé. Le tableau met en évidence d'un seul coup d'œil la responsabilité de chaque classe et la manière dont elles interagissent.

5.1 Connexion à la base de données – Database.java

1. Contexte

La classe Database est le point d'entrée unique vers la base MySQL. Toute classe ayant besoin de données passe par elle pour obtenir une connexion. Elle ne travaille pas seule plusieurs classes gravitent autour d'elle.

1.a Classes concernées et leur rôle précis :

Classe / Fichier	Rôle précis dans cette partie
Database	Gère la connexion JDBC selon le patron Singleton : lecture de la configuration, ouverture, test de disponibilité et fermeture propre de la connexion.
database.properties	Fichier de configuration externe (ressource) fournissant l'URL, l'utilisateur et le mot de passe de connexion.
DAO<T>, ChambreDAO, ReservationDAO	Principaux consommateurs : ils obtiennent la connexion partagée via Database.getInstance().getConnection() pour exécuter leurs requêtes.
MainApp	Teste la connexion au démarrage (testerConnexion) et la ferme à l'arrêt de l'application (close).
AlertUtil	Affiche un avertissement clair à l'utilisateur si la base est injoignable au lancement.

2. Fonctionnement

Au lancement, MainApp interroge Database pour vérifier que la base est accessible en cas d'échec, AlertUtil prévient l'utilisateur sans bloquer l'ouverture de l'application. Database lit d'abord les identifiants depuis database.properties (avec des valeurs de repli si le fichier est absent), puis la méthode getConnection() charge le pilote JDBC de MySQL et ouvre la connexion si elle n'existe pas encore ou si elle a été fermée. Toutes les classes DAO réutilisent ensuite cette même connexion.

3. Choix architecturaux

Nous avons implémenté Database en Singleton afin qu'une seule connexion soit partagée pendant toute la durée de vie du programme : ce choix est adapté à une application de bureau mono-utilisateur et évite de rouvrir une connexion à chaque opération. Les classes DAO ne dépendent que de cette abstraction, sans jamais manipuler elles-mêmes les paramètres de connexion. Externaliser ces paramètres dans un fichier de propriétés permet enfin de changer de serveur ou de mot de passe sans recompiler le code.

```
public static synchronized Database getInstance() {
    if (instance == null) instance = new Database();
    return instance;
}

public Connection getConnection() throws SQLException {
    if (connection == null || connection.isClosed()) {
        Class.forName("com.mysql.cj.jdbc.Driver");
        connection = DriverManager.getConnection(url, user, password);
    }
    return connection;
}

public boolean testerConnexion() {           // appelé par MainApp au démarrage
    try { return getConnection() != null && getConnection().isValid(3); }
    catch (SQLException e) { return false; }
}
```

5.2 Les classes modèles

1. Contexte

Les classes modèles représentent les objets du monde réel manipulés par l'application : une Chambre et une Réservation. Ce sont des POJO (Plain Old Java Objects), enrichis de quelques énumérations et de getters « calculés ».

1.a Classes concernées et leur rôle précis :

Classe / Fichier	Rôle précis dans cette partie
Chambre	Entité 1 : numéro, type, prix par nuit, capacité, étage, disponibilité, description ; expose <code>getEtatLibelle()</code> (« Disponible » / « Occupée ») pour l'affichage.
Reservation	Entité 2 : informations client, dates, statut, montant, paiement, et lien vers la chambre (<code>chambreId + chambreNumero</code>) ; expose <code>getNombreNuits()</code> et <code>getPaiementLibelle()</code> .
TypeChambre (enum)	Types de chambres (Simple, Double, Suite, Familiale, Deluxe), chacun portant un libellé lisible et une capacité par défaut.
StatutReservation (enum)	États possibles d'une réservation : En attente, Confirmée, Annulée, Terminée.
DAO (consommateurs)	Convertissent les lignes de la base en objets modèles et inversement (mapping).
Contrôleurs (consommateurs)	Affichent ces objets dans les TableView et ComboBox via les getters JavaBean.

2. Fonctionnement

Chaque attribut d'un modèle correspond à une colonne de la base. Les deux énumérations garantissent que seules des valeurs valides circulent dans l'application. Les getters calculés évitent de dupliquer de la logique dans l'interface : par exemple `getNombreNuits()` déduit la durée du séjour à partir des deux dates, et `getEtatLibelle()` renvoie directement le texte affiché dans le tableau des chambres.

3. Choix architecturaux

Le respect strict de la convention JavaBean (`getXxx`) n'est pas accessoire : il permet aux colonnes de TableView, via `PropertyValueFactory`, de lire automatiquement la valeur à afficher à partir du nom de la propriété. Recourir à des énumérations plutôt qu'à de simples chaînes de caractères élimine les fautes de frappe et centralise les libellés. Les modèles restent volontairement « passifs » (aucune logique d'accès aux données) afin de respecter la séparation des couches.

```
public enum TypeChambre { SIMPLE, DOUBLE, SUITE, FAMILIALE, DELUXE; }
public enum StatutReservation { EN_ATTENTE, CONFIRMEE, ANNULEE, TERMINEE; }

// Getters « calculés » exploités directement par l'interface
public long getNombreNuits() {
    if (dateArrivee == null || dateDepart == null) return 0;
    return Math.max(ChronoUnit.DAYS.between(dateArrivee, dateDepart), 0);
}

public String getEtatLibelle() { return disponible ? "Disponible" : "Occupée"; }
```

5.3 Les classes DAO

1. Contexte

Les DAO (Data Access Object) forment la couche qui « parle » SQL. Elles isolent complètement les requêtes vers la base du reste de l'application : un contrôleur demande simplement « donne-moi toutes les chambres » sans savoir comment elles sont stockées.

1.a Classes concernées et leur rôle précis :

Classe / Fichier	Rôle précis dans cette partie
DAO<T> (interface)	Définit le contrat CRUD commun à toutes les entités : findAll(), findById(), insert(), update(), delete().
ChambreDAO	Implémente DAO<Chambre> ; ajoute des méthodes métier : findDisponibles() et compterParType() (statistiques).
ReservationDAO	Implémente DAO<Reservation> ; ses lectures utilisent un LEFT JOIN pour récupérer le numéro de chambre ; ajoute compterParStatut().
Database	Fournit la connexion partagée utilisée par les deux DAO.
Chambre, Reservation	Objets produits (lecture) et consommés (écriture) par les DAO via une méthode mapper().

2. Fonctionnement

Nous avons d'abord défini une interface générique DAO<T> qui fixe les opérations CRUD. Chaque entité possède ensuite son DAO concret. Une méthode privée mapper() convertit une ligne de résultat (ResultSet) en objet Java. Les lectures de ReservationDAO s'appuient sur une requête de base (SELECT_BASE) contenant une jointure, afin de ramener en une seule fois le numéro de la chambre associée à chaque réservation. Les méthodes compterParType() et compterParStatut() alimentent directement les graphiques du tableau de bord.

3. Choix architecturaux

Trois décisions structurantes :

- l'usage systématique de PreparedStatement (requêtes paramétrées) protège contre les injections SQL et gère correctement les types ;
- la jointure dans ReservationDAO évite le problème « N+1 » — afficher 50 réservations ne coûte qu'une seule requête au lieu de 51 ;
- l'interface générique factorise le contrat commun et rend le code homogène d'une entité à l'autre, ce qui facilite l'ajout futur d'une nouvelle entité.

```
public interface DAO<T> {                                // contrat commun à toutes les entités
    List<T> findAll();
    Optional<T> findById(int id);
    boolean insert(T t);
    boolean update(T t);
    boolean delete(int id);
}
```

```
// ReservationDAO : une seule requête grâce à la jointure (évite le N+1)
private static final String SELECT_BASE =
    "SELECT r.*, c.numero AS chambre_numero " +
    "FROM reservations r " +
    "LEFT JOIN chambres c ON r.chambre_id = c.id ";

// ChambreDAO : insertion sécurisée + récupération de la clé générée
public boolean insert(Chambre c) {
    String sql = "INSERT INTO chambres (numero, type, prix_nuit, capacite,"
        + " etage, disponible, description) VALUES (?, ?, ?, ?, ?, ?, ?)";
    try (PreparedStatement ps = db.getConnection()
        .prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {
        remplirParametres(ps, c);
        if (ps.executeUpdate() == 0) return false;
        try (ResultSet keys = ps.getGeneratedKeys()) {
            if (keys.next()) c.setId(keys.getInt(1));
        }
        return true;
    } catch (SQLException e) { return false; }
}
```

5.4 Les contrôleurs

1. Contexte

Les contrôleurs sont le cœur dynamique de l'application : ils font le lien entre ce que voit l'utilisateur (la vue FXML) et ce que fait le programme (les modèles et les DAO). Chaque vue possède son contrôleur.

1.1 Classes concernées et leur rôle précis :

Classe / Fichier	Rôle précis dans cette partie
MainController	Orchestre la navigation entre les sections (barre latérale / TabPane) et synchronise les données : après l'ajout d'une chambre, la liste des chambres disponibles côté réservations est rafraîchie.
ChambreController	CRUD des chambres, recherche/filtrage en temps réel, mise en forme des colonnes et export CSV.
ReservationController	CRUD des réservations liées à une chambre, calcul automatique du montant, validation des saisies et génération de facture.
DashboardController	Calcule les indicateurs, alimente les graphiques, génère les analyses et gère la personnalisation.
ChambreDAO, ReservationDAO	Sollicités par les contrôleurs pour lire et écrire les données.
AlertUtil, CSVExporter, ThemeManager	Services transverses appelés par les contrôleurs (dialogues, export, thème).

2. Fonctionnement

À l'ouverture d'une vue, la méthode `initialize()` (annotée `@FXML`) prépare l'écran : configuration des colonnes, remplissage des listes déroulantes, branchement des filtres et chargement initial des données. Dans `ReservationController`, des écouteurs (listeners) recalculent le montant total dès que l'utilisateur change la chambre ou les dates. La recherche et le filtrage reposent sur une `FilteredList` couplée à une `SortedList`, qui met à jour le tableau instantanément, sans relancer de requête vers la base.

3. Choix architecturaux

La validation des saisies (champs obligatoires, format d'e-mail par expression régulière, cohérence des dates, capacité de la chambre) est centralisée dans une méthode dédiée appelée avant tout enregistrement : seules des données valides atteignent la base. Cette organisation respecte le principe de responsabilité unique. La vue affiche, le contrôleur décide, le DAO persiste ainsi délègue aux classes utilitaires tout ce qui est transverse.

```
// Recherche + tri en temps réel, sans requête SQL supplémentaire
FilteredList<Reservation> filtre = new FilteredList<>(donnees, r -> true);
tfRecherche.textProperty().addListener((o, a, n) -> appliquerFiltres(filtre));
cbFiltreStatut.valueProperty().addListener((o, a, n) -> appliquerFiltres(filtre));
SortedList<Reservation> trie = new SortedList<>(filtre);
trie.comparatorProperty().bind(tableReservations.comparatorProperty());
tableReservations.setItems(trie);

// Recalcul automatique du montant à chaque changement pertinent
cbChambre.valueProperty().addListener((o, a, n) -> recalculerMontant());
dpArrivee.valueProperty().addListener((o, a, n) -> recalculerMontant());
dpDepart.valueProperty().addListener((o, a, n) -> recalculerMontant());

// Validation centralisée : la base ne reçoit que des données valides
private boolean validerSaisie() {
    StringBuilder erreurs = new StringBuilder();
    if (tfNom.getText() == null || tfNom.getText().trim().isEmpty())
        erreurs.append("• Le nom du client est obligatoire.\n");
    if (cbChambre.getValue() == null)
        erreurs.append("• Vous devez sélectionner une chambre.\n");
    if (!dpDepart.getValue().isAfter(dpArrivee.getValue()))
        erreurs.append("• La date de départ doit suivre l'arrivée.\n");
    if (spPersonnes.getValue() > cbChambre.getValue().getCapacite())
        erreurs.append("• Nombre de personnes supérieur à la capacité.\n");
    if (erreurs.length() > 0) {
        AlertUtil.avertissement("Saisie incomplète", erreurs.toString());
        return false;
    }
    return true;
}
```

5.5 Les fichiers FXML

1. Contexte

Les fichiers FXML décrivent l'interface graphique de manière déclarative, un peu comme une page web décrit sa structure en HTML. JavaFX charge ces fichiers et les relie automatiquement aux contrôleurs.

1.a Classes concernées et leur rôle précis :

Classe / Fichier	Rôle précis dans cette partie
<code>MainView.fxml</code>	Structure générale (BorderPane + barre latérale + zone centrale) ; déclare <code>fx:controller = MainController</code> .
<code>ChambreView.fxml</code>	Formulaire de saisie et TableView des chambres ; piloté par <code>ChambreController</code> .
<code>ReservationView.fxml</code>	Formulaire de réservation et TableView associé ; piloté par <code>ReservationController</code> .
<code>DashboardView.fxml</code>	Indicateurs, graphiques (PieChart, BarChart) et panneau de personnalisation ; piloté par <code>DashboardController</code> .
<code>styles.css</code>	Feuille de style commune à toutes les vues, pilotée par la couleur dynamique <code>-app-accent</code> .

2. Fonctionnement

Chaque fichier FXML agence les composants à l'aide de conteneurs : `BorderPane` pour la structure générale, `VBox` et `HBox` pour empiler ou aligner, `GridPane` pour les formulaires. Chaque composant utile reçoit un identifiant `fx:id` relié à l'attribut correspondant du contrôleur, et les boutons déclarent l'action à exécuter via `onAction`. L'attribut `fx:controller` indique quel contrôleur pilote la vue. Les colonnes de tableau sont liées aux propriétés des modèles via `PropertyValueFactory`.

3. Choix architecturaux

Séparer la description de l'interface (FXML) de sa logique (Java) est précisément ce que recommande le patron MVC : les écrans deviennent plus lisibles, modifiables visuellement avec `Scene Builder`, et leur apparence peut évoluer sans toucher au code métier. Toute la mise en forme est déléguée à une feuille de style CSS unique, ce qui garantit une identité visuelle cohérente et un thème modifiable à chaud.

```
<!-- MainView.fxml : structure générale reliée à son contrôleur -->
<BorderPane fx:controller="ma.ensao.hotel.controller.MainController">
    <left>    <VBox styleClass="sidebar"> ... </VBox> </left>
    <center> <TabPane fx:id="tabs"> ... </TabPane> </center>
</BorderPane>

// Côté contrôleur : liaison d'une colonne à une propriété du modèle
colNumero.setCellValueFactory(new PropertyValueFactory<>("numero"));
colEtat.setCellValueFactory(new PropertyValueFactory<>("etatLibelle"));
```

5.6 Fonctionnalité spécifique au projet

Au-delà des fonctionnalités exigées par le sujet, nous avons développé plusieurs fonctionnalités originales qui distinguent notre application. Plutôt que de les présenter en bloc, nous les détaillons une à une, en précisant pour chacune les classes mises en jeu et le code essentiel.

5.6.1 Génération et export de factures

1. Classes concernées :

ReservationController (méthodes handleFacture, construireFacture, enregistrerFacture, numeroFacture), le modèle Reservation, ainsi que FileChooser et FileWriter pour l'enregistrement.

2. Fonctionnement

Depuis l'écran des réservations, un bouton « Facture » construit un document récapitulatif (numéro de facture, client, chambre, dates, nombre de nuits, montant) mis en forme dans une zone de texte à police fixe. L'utilisateur peut ensuite enregistrer cette facture sur le disque via un FileChooser. Le numéro de facture est généré à partir de l'identifiant de la réservation et de la date courante.

```
private void handleFacture() {
    Reservation r = tableReservations.getSelectionModel().getSelectedItem();
    if (r == null) { AlertUtil.info("Facture", "Sélectionnez une réservation.");
    return; }
    String contenu = construireFacture(r);           // texte formaté
    FileChooser fc = new FileChooser();
    fc.setInitialFileName(numeroFacture(r) + ".txt");
    File f = fc.showSaveDialog(tableReservations.getScene().getWindow());
    if (f != null) enregistrerFacture(f, contenu); // écriture via FileWriter
}
```

5.6.2 Personnalisation de la couleur d'accent en direct

1. Classes concernées

ThemeManager (appliquerAccent, versHex), DashboardController (le ColorPicker cpTheme), le nœud racine de la scène et la feuille styles.css qui référence la couleur dynamique -app-accent.

2. Fonctionnement

Un ColorPicker situé sur le tableau de bord permet de choisir une couleur d'accent. ThemeManager injecte cette couleur comme looked-up color CSS (-app-accent) sur le nœud racine. Toutes les règles CSS qui référencent -app-accent (barre latérale, boutons, graphiques) se mettent alors à jour automatiquement, et la couleur se propage à tous les enfants — sans recharger l'interface.

3. Choix architecturaux

Regrouper cette logique dans une classe utilitaire ThemeManager évite de dupliquer du code de style dans chaque contrôleur. Le recours aux looked-up colors illustre une bonne pratique : on définit une couleur « variable » en un seul endroit du CSS, et il suffit de la modifier pour recolorer toute l'interface


```
// ThemeManager : une seule ligne recolore toute l'application
public static void appliquerAccent(Parent racine, Color couleur) {
    if (racine == null || couleur == null) return;
    racine.setStyle("-app-accent: " + versHex(couleur) + ";");
}
public static String versHex(Color c) {
    return String.format("#%02X%02X%02X",
        (int) Math.round(c.getRed() * 255),
        (int) Math.round(c.getGreen() * 255),
        (int) Math.round(c.getBlue() * 255));
}
```

5.6.3 Mode clair / sombre

1. Classes concernées

MainController (bascule du thème), le nœud racine BorderPane et la feuille styles.css (jeu de couleurs alternatif sous la classe .dark).

2. Fonctionnement

Un bouton bascule l'interface entre un mode clair et un mode sombre. Techniquement, il suffit d'ajouter ou de retirer la classe CSS .dark sur le nœud racine : la feuille de style définit, sous .dark, un jeu de couleurs inversé (fonds, textes, surfaces). Toute l'application change instantanément de palette.

```
private void basculerThemeSombre() {
    var classes = racine.getStyleClass();
    if (classes.contains("dark")) classes.remove("dark");
    else classes.add("dark");
}
```

5.6.4 Compteurs animés du tableau de bord

1. Classes concernées

DashboardController (méthode animerValeur), associée aux classes d'animation JavaFX Timeline, KeyFrame, KeyValue et à une SimpleDoubleProperty.

2. Fonctionnement

À l'affichage du tableau de bord, chaque indicateur (nombre de chambres, revenu, taux d'occupation...) ne s'affiche pas brutalement : il s'incrémente de zéro jusqu'à sa valeur finale en 700 ms, grâce à une animation pilotée par une propriété numérique observée. La barre de progression du taux d'occupation est animée de la même façon.

```
// Effet « compteur » : la valeur s'anime de 0 jusqu'à la cible
private void animerValeur(Label label, double cible, String format) {
    DoubleProperty valeur = new SimpleDoubleProperty(0);
    valeur.addListener((o, a, n) ->
        label.setText(String.format(format, n.doubleValue())));
    new Timeline(
        new KeyFrame(Duration.ZERO, new KeyValue(valeur, 0)),
        new KeyFrame(Duration.millis(700), new KeyValue(valeur, cible))
    ).play();
}
```

5.6.5 Analyses intelligentes

1. Classes concernées

DashboardController (méthode chargerInsights), les modèles Chambre et Reservation, et l'énumération StatutReservation.

2. Fonctionnement

À partir des données existantes, l'application génère automatiquement des recommandations affichées dans une liste : niveau d'occupation (avec conseil tarifaire), nombre de réservations en attente, type de chambre le plus rentable, etc. Ces analyses transforment de simples chiffres en informations utiles à la décision.

```
double taux = total == 0 ? 0 : (double) occupees / total;
if (taux >= 0.8)
    insights.add("Occupation élevée – envisagez d'augmenter vos tarifs.");
else if (taux <= 0.3)
    insights.add("Forte disponibilité – moment idéal pour des promotions.");

revParType.entrySet().stream()                // type le plus rentable
    .max(Map.Entry.comparingByValue())
    .ifPresent(e -> insights.add("Type le plus rentable : "
        + e.getKey().getLibelle()));
```

Chapitre 6 Interfaces de l'application

Ce chapitre présente les principales interfaces de HotelManager.

6.1 Fenêtre principale

La fenêtre principale s'articule autour d'une barre de navigation latérale donnant accès aux trois sections (Chambres, Réservations, Tableau de bord), d'un en-tête présentant le titre et l'horloge, et d'une zone centrale qui affiche la section active.

The screenshot shows the main interface of HotelManager. On the left is a dark blue sidebar with the logo 'Maison Azur ENSAO • GI3' and a navigation menu with 'Chambres', 'Reservations', and 'Tableau de bord'. The top header is blue with 'Fichier', 'Affichage', and 'Aide' menus, and a welcome message 'Bonjour, bienvenue samedi 20 juin 2026' with a clock showing '04:24:20'. The main content area is titled 'Fiche chambre' and contains a search bar, a table of rooms, and buttons for 'Ajouter', 'Modifier', 'Supprimer', and 'Vider'. The table lists 11 rooms with columns for ID, Numéro, Type, Prix / nuit, Capacité, Étage, and État.

ID	Numéro	Type	Prix / nuit	Capacité	Étage	État
1	101	Simple	306,04 MAD	4	1	Occupée
2	102	Simple	350,00 MAD	1	1	Occupée
3	201	Double	550,00 MAD	2	2	Disponible
4	202	Double	550,00 MAD	2	2	Disponible
11	203	Double	406,59 MAD	4	4	Disponible
5	301	Familiale	850,00 MAD	4	3	Disponible
6	302	Familiale	850,00 MAD	4	3	Occupée
7	401	Suite	1200,00 MAD	3	4	Disponible
8	402	Suite	1200,00 MAD	3	4	Disponible
9	501	Deluxe	1800,00 MAD	2	5	Disponible
10	502	Deluxe	1800,00 MAD	2	5	Occupée

Figure 6.1 — Fenêtre principale de l'application

6.2 Interfaces de gestion de l'entité 1 (Chambres)

Cet écran réunit le formulaire de saisie d'une chambre (numéro, type, prix, capacité, étage, disponibilité, description) et le tableau récapitulatif listant toutes les chambres, avec recherche et filtre par type.

This screenshot shows the 'Fiche chambre' management interface. It features a sidebar on the left with 'Chambres', 'Reservations', and 'Tableau de bord'. The top header is blue with 'Fichier', 'Affichage', and 'Aide' menus, and a welcome message 'Bonjour, bienvenue samedi 20 juin 2026' with a clock showing '04:24:20'. The main content area is titled 'Fiche chambre' and contains a search bar, a table of rooms, and buttons for 'Ajouter', 'Modifier', 'Supprimer', and 'Vider'. The table lists 11 rooms with columns for ID, Numéro, Type, Prix / nuit, Capacité, Étage, and État.

ID	Numéro	Type	Prix / nuit	Capacité	Étage	État
1	101	Simple	306,04 MAD	4	1	Occupée
2	102	Simple	350,00 MAD	1	1	Occupée
3	201	Double	550,00 MAD	2	2	Disponible
4	202	Double	550,00 MAD	2	2	Disponible
11	203	Double	406,59 MAD	4	4	Disponible
5	301	Familiale	850,00 MAD	4	3	Disponible
6	302	Familiale	850,00 MAD	4	3	Occupée
7	401	Suite	1200,00 MAD	3	4	Disponible
8	402	Suite	1200,00 MAD	3	4	Disponible
9	501	Deluxe	1800,00 MAD	2	5	Disponible
10	502	Deluxe	1800,00 MAD	2	5	Occupée

Figure 6.2.1 — Gestion des chambres

Maison Azur

ENSAO • GI3

NAVIGATION

Chambres

Reservations

Tableau de bord

Astuce

Changez la couleur d'accent depuis le tableau de bord : tout l'habillage s'adapte.

© 2025-2026 • Java IHM

Fiche chambre

Informations générales

Tarif & disponibilité

Prix / nuit :

0 MAD

0 1000 2000 3000 4000 5000

☒ Chambre disponible

Description :

Équipements, vue, remarques...

Ajouter

Modifier

Supprimer

Vider

Rechercher (numéro, desc)

Type : Tous

Réinitialiser

Exporter CSV

ID	Numéro	Type	Prix / nuit	Capacité	Étage	État
1	101	Simple	306,04 MAD	4	1	Occupée
2	102	Simple	350,00 MAD	1	1	Occupée
3	201	Double	550,00 MAD	2	2	Disponible
4	202	Double	550,00 MAD	2	2	Disponible
11	203	Double	406,59 MAD	4	4	Disponible
5	301	Familiale	850,00 MAD	4	3	Disponible
6	302	Familiale	850,00 MAD	4	3	Occupée
7	401	Suite	1200,00 MAD	3	4	Disponible
8	402	Suite	1200,00 MAD	3	4	Disponible
9	501	Deluxe	1800,00 MAD	2	5	Disponible
10	502	Deluxe	1800,00 MAD	2	5	Occupée

11 chambre(s) affichée(s)

Figure 6.2.2 — Gestion des chambres

6.3 Interfaces de gestion de l'entité 2 (Réservations)

Cet écran permet de créer une réservation liée à une chambre. Le montant total se calcule automatiquement en fonction des dates et du prix de la chambre. Un filtre par statut et la génération de facture y sont disponibles.

Maison Azur

ENSAO • GI3

NAVIGATION

Chambres

Reservations

Tableau de bord

Astuce

Changez la couleur d'accent depuis le tableau de bord : tout l'habillage s'adapte.

© 2025-2026 • Java IHM

Fiche réservation

Client & chambre

Séjour

Statut & paiement

Client :

Nom complet

Email :

client@email.com

Téléphone :

06 00 00 00 00

Chambre :

Choisir une chambre

Ajouter

Modifier

Supprimer

Vider

Rechercher (client, ch)

S...

Tous

Réinit...

F...

Export...

ID	Client	Chambre	Arrivée	Départ	Nuits	Statut	Montant	Paiement
5	Omar Haddad	501	2026-07-...	2026-07-...	4	En attente	7200,00 M...	Non payé
4	Laura Schmidt	202	2026-07-...	2026-07-...	4	En attente	2200,00 M...	Non payé
7	James Carter	402	2026-06-...	2026-06-...	2	Confirmée	2400,00 M...	Non payé
2	Sofia Marlowe	401	2026-06-...	2026-06-...	4	Confirmée	4800,00 M...	Payé
1	Karim Benali	201	2026-06-...	2026-06-...	3	Confirmée	1650,00 M...	Payé
3	Yassine El Amr...	101	2026-06-...	2026-06-...	1	Terminée	350,00 MAD	Payé
6	Fatima Zahra	301	2026-06-...	2026-06-...	3	Annulée	2550,00 M...	Non payé

7 réservation(s) affichée(s)

Figure 6.3.1 — Gestion des réservations

Fiche réservation

Client & chambre

Séjour

Arrivée : 20/06/2026

Départ : 21/06/2026

Personnes : 1

Statut & paiement

Statut de la réservation :

☒ En attente
 ☐ Confirmée
 ☐ Annulée

☐ Terminée

☐ Réservation payée

Montant total : 0.00 MAD

Remarques :

Demands particulières, notes internes...

Figure 6.3.2 — Gestion des réservations

Figure 6.3.3 — Gestion des réservations

6.4 Tableau de bord – Statistiques

Le tableau de bord présente les indicateurs clés (nombre de chambres, disponibilités, chiffre d'affaires), le taux d'occupation sous forme de barre de progression et des graphiques de répartition (par type de chambre et par statut de réservation).



Figure 6.4.1 — Tableau de bord et statistiques



Figure 6.4.2 — Tableau de bord et statistiques

6.5 Menus et dialogues

La barre de menus structure les actions de l'application, tandis que les boîtes de dialogue (Alert) confirment les opérations et signalent les erreurs. Des info-bulles guident l'utilisateur lors de la saisie.

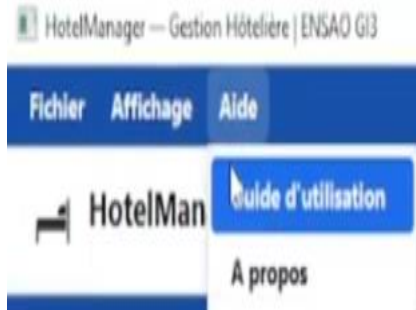


Figure 6.5.1— Menus et dialogues

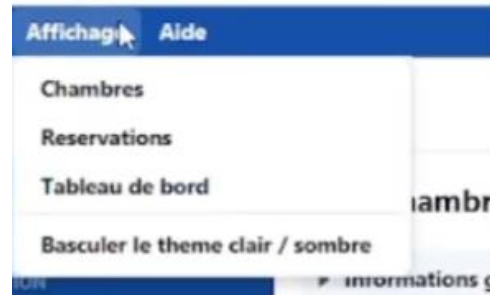


Figure 6.5.2— Menus et dialogues



Figure 6.5.3— Menus et dialogues

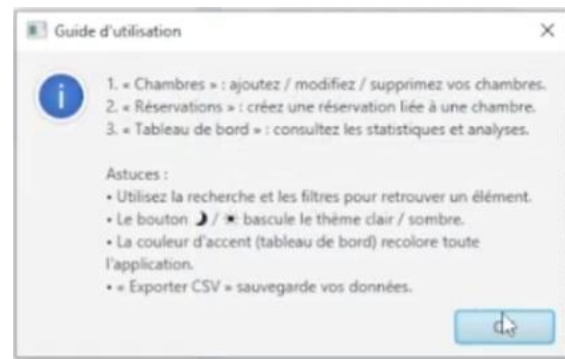


Figure 6.5.4— Menus et dialogues

6.6 Export de données

L'export CSV s'effectue via un FileChooser : l'utilisateur choisit l'emplacement et le nom du fichier, puis les données du tableau courant sont enregistrées au format CSV, réutilisable dans un tableau.

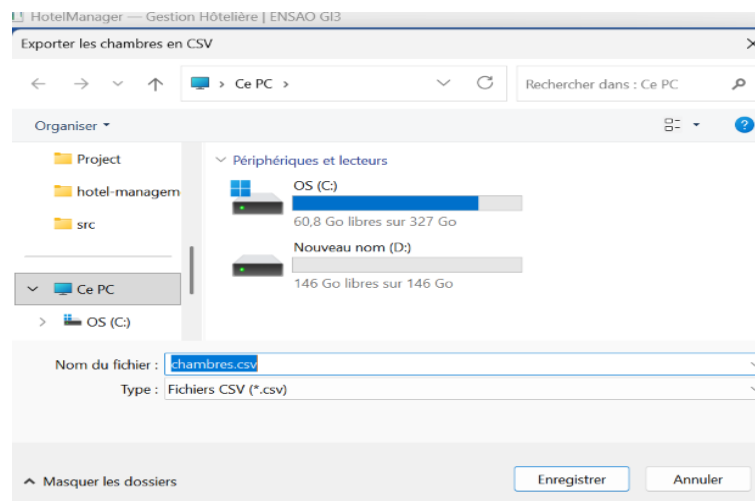
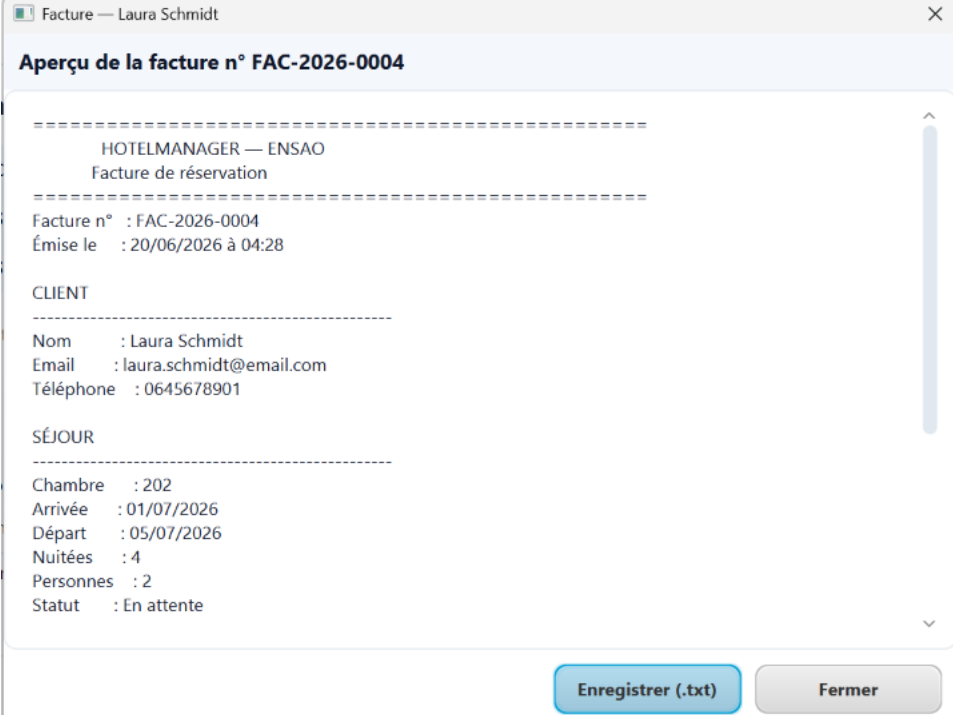


Figure 6.6 — Export des données au format CSV

6.7 Génération de facture

Depuis l'écran des réservations, un bouton « Facture » construit un document récapitulatif (numéro de facture, client, chambre, dates, nombre de nuits, montant) mis en forme dans une zone de texte à police fixe.



The screenshot shows a window titled 'Facture — Laura Schmidt'. The main content area is titled 'Aperçu de la facture n° FAC-2026-0004'. It contains a text-based summary of a reservation, formatted with dashed lines as section separators. The summary includes the hotel name, invoice number, issue date, client details (name, email, phone), and stay details (room, arrival, departure, nights, number of people, and status). At the bottom right, there are two buttons: 'Enregistrer (.txt)' and 'Fermer'.

Facture — Laura Schmidt

Aperçu de la facture n° FAC-2026-0004

=====

HOTELMANAGER — ENSAO
Facture de réservation

=====

Facture n° : FAC-2026-0004
Émise le : 20/06/2026 à 04:28

CLIENT

Nom : Laura Schmidt
Email : laura.schmidt@email.com
Téléphone : 0645678901

SÉJOUR

Chambre : 202
Arrivée : 01/07/2026
Départ : 05/07/2026
Nuitées : 4
Personnes : 2
Statut : En attente

Enregistrer (.txt) Fermer

Figure 6.7— Génération d'une facture

Conclusion

Ce mini-projet nous a permis de concevoir et de développer, de bout en bout, une application de gestion hôtelière complète et fonctionnelle. En partant du choix du domaine jusqu'à la livraison finale, nous avons mis en pratique l'ensemble des notions abordées dans le module : la construction d'interfaces graphiques avec JavaFX, l'organisation du code selon l'architecture MVC associée au pattern DAO, et la connexion à une base de données MySQL via JDBC.

Au-delà de l'aspect technique, ce projet a été une véritable expérience de travail en équipe. La répartition équilibrée des tâches, la communication régulière et l'usage d'un outil de gestion de versions nous ont appris à collaborer de manière professionnelle et organisée. Nous avons également développé notre rigueur, notamment lors des phases de validation des saisies et de test des cas limites.

L'application répond à l'ensemble des exigences fonctionnelles demandées et offre, en complément, des fonctionnalités originales telles que la génération de factures, la personnalisation du thème et un mode sombre. Plusieurs pistes d'amélioration restent envisageables : la gestion de plusieurs utilisateurs avec authentification, l'ajout d'un module de facturation plus avancé, la détection automatique des conflits de réservation, ou encore le déploiement de l'application sous forme d'exécutable autonome.

En définitive, ce travail constitue une base solide et représentative de ce que peut produire une petite équipe de développement, et il nous laisse une compréhension concrète et durable du développement d'applications de bureau connectées à une base de données.

Références

- Documentation officielle JavaFX : <https://openjfx.io>
- Documentation MySQL : <https://dev.mysql.com/doc/>
- Documentation Java SE 17 (Oracle) : <https://docs.oracle.com/en/java/javase/17/>
- Tutoriel JDBC (Oracle) : <https://docs.oracle.com/javase/tutorial/jdbc/>
- Apache Maven : <https://maven.apache.org/guides/>
- Pilote MySQL Connector/J : <https://dev.mysql.com/doc/connector-j/en/>